

에이전트 레일 파이프라인

초록

본 문서는 개인 개발자가 소프트웨어 태스크를 backlog 계층(마일스톤 · 태스크 · AC) 단위로 분해하고, 태스크마다 동적으로 생성·회수되는 4개의 역할형 에이전트(Implementer · Reviewer · Verifier · Logger)로 실행하는 오케스트레이션 아키텍처를 제안한다. 이 설계는 에이전트를 왜 · 언제 · 어떻게 · 얼마나 투입해야 하는가라는 네 가지 질문에서 출발했으며, 각 질문에 대한 답을 실제 근거로 뒷받침한다. backlog.md는 오픈소스 CLI를 그대로 채택한 것이고, claude-rails와 git-format은 이 오케스트레이션을 위해 사전에 직접 구축한 도구다. 본문이 인용하는 외부 문헌 조사는 현재도 계속 진행 중이다. 주요 결론은 다음과 같다 — 멀티에이전트 구조는 처리량이 아니라 판단의 독립성을 위해 채택되고, 그 정당성은 단일 에이전트의 기준 성공률이 낮은 복잡한 작업에 한정된다. 태스크 간 실행 순서는 의존성과 파일 결합도로 정하고 그 그래프의 길이(임계 경로)와 폭(동시 배정 가능 태스크 수)을 세어 동시성 상한의 근거로 삼으며, 태스크 내부에서는 구현은 순차적이되 그 결과에 대한 판단은 병렬로 이루어지고, 이 병렬 판단은 부착 시 캐싱한 문서 프리픽스를 공유해 비용을 억제한다. 에이전트의 수 · 역할 · 시점 · 모델은 오케스트레이터 내부에 고정하지 않고 외부에서 주입되는 정책으로 결정한다. 문서 전반에서 확정된 결정과 미해결 질문을 구분해 명시했으며, 인용은 모두 각주로 정리했다.

• DRAFT · 검증 대기

2026-09-21 정리본 · 정독 30~40분

○ 요약	○ 00	○ Phase A	○ Phase B	○ Phase C	○ 06
결론부터	왜 멀티에이전트	부트스트랩	백로깅	실행	오케스트레이터
○ 07	○ 09	○ 10	○ 11	○ 12	○ 14
역할표	미해결	실사례	트레이드오프	구현가능성	적용범위
○ 각주					
인용·출처					

요약 결론 — 그래서 어떻게 작업하는가

1. **지금 당장:** 4-role 전체가 아니라 Implementer 혼자만 스폰 → 작업 → 토큰 집계(git-format 방식 재사용) → /clear → 회수하는 walking skeleton부터 만든다. 아직 없는 Reviewer/Verifier 채점 기준을 요구하지 않는 유일한 조각이다 (12절).
2. **판단은 프로젝트 종류가 아니라 그 순간의 태스크 난이도로 한다:** 단일 에이전트가 이미 안정적으로 해내는 태스크에는 멀티에이전트를 쓰지 않는다. Next.js 스캐폴드나 Spring+DB 단순 CRUD처럼 단순하게 시작한 프로젝트라도, 이후 태스크의 난이도가 올라가면 그 시점부터 4-role 전체를 쓴다 — 판단은 프로젝트 단위가 아니라 태스크마다 반복한다. 경계선 자체는 아직 없고, 직접 측정해서 정한다 (14절).
3. **부트스트랩 순서는 고정:** git init → backlog.md CLI → backlog init → git-format → claude-rails → (필요 시) langtoolchain으로 Java 25 설치 (02절).
4. **백로그는 한 번에 다 만들고, 이후에도 반복 재진입:** 브리핑 → plan mode → draft → 승인 → promote. 진행 중 새 마일스톤이 필요해지면 언제든지 이 루프로 돌아온다 (03절).
5. **병렬처리하는 두 신호로 정한다:** 선언된 의존성 그래프와, 같은 파일/모듈을 건드리는지(결합도)를 함께 본다 — 의존성이 없어도 결합돼 있으면 순차로 다룬다. 순차 체인은 워커풀의 빈 슬롯을 막아선 안 되고, 동시에 도는 에이전트 수를 무한정 늘리지도 않는다 (04-0절).
6. **4-role이 필요한 태스크는:** Implementer가 AC를 하나씩 구현·커밋할 때마다 Reviewer(판단)와 Verifier·DoD(결정적 검사)가 서로 기다리지 않고 동시에 그 커밋을 본다 — 전부 부착 시 캐싱해둔 같은 조상 문서를 공유해서 저렴하다. 전체 AC가 끝나면 Verifier·AC가 마지막 판정을 하고, Logger는 그 사이 내내 관찰한다. 전부 독립 프로세스이고, Java 25 + virtual thread + Semaphore 오케스트레이터가 외부에서 정책을 주입한다 (04·06·07·11절).
7. **더 만들기 전에 짧게 실측해서 확인할 것:** 헤드리스 세션 간 메시징 실행 여부, backlog.md의 의존성/마일스톤/상태 필드 실제 지원 여부, claude 프로세스 peak RSS와 API 플랜의 분당 한도 (09절).
8. **PKM 완성 전까지 채우지 않는 것:** Reviewer의 pass/fail 기준, Verifier·AC 판정 기준 — 추측으로 채우지 않는다 (11절).

00 왜 멀티에이전트인가

이 RFC는 방어할 결론을 미리 정해두고 근거를 모은 게 아니다. 출발점은 단순한 호기심이었다 — 에이전트 모델을 왜 · 언제 · 어떻게 · 얼마나 써야 하는가. 그 네 질문을 실제로 파고든 결과가 이 문서 전체다.

질문	지금까지의 답	어디서
왜 쓰는 가	처리량이 아니라 판단의 독립성 — Reviewer/Verifier가 Implementer의 추론을 상속받지 않게 컨텍스트 격리 자체를 게이트로 쓴다	이 절 (00)

질문	지금까지의 답	어디서
언제 쓰는가	단일 에이전트가 이미 안정적으로 해내는 작업에는 쓰지 않는다 (경계선은 측정 대상, 14절). 여러 태스크가 동시에 Ready면 의존성·파일 결합이 있는 것만 순차, 나머지는 병렬	04-0 · 14절
어떻게 쓰는가	Implementer↔Reviewer↔Verifier↔Logger를 완전히 독립된 프로세스로, JVM 오케스트레이터가 정책을 외부에서 주입해 스폰·회수	04 · 06 · 07절
얼마나 쓰는가	몇 개·어떤 역할·어떤 모델을 붙일지는 Allocation Policy가 노드+상태를 보고 결정. 용량 상한은 벤치마크가 아니라 min(그래프 폭, 자원 실측(peak RSS·API 한도))으로 정한다 — 순차 그래프의 길이(임계 경로)와 폭(동시 배정 가능 태스크 수)은 스케줄 라운드마다 세어 기록한다	06절

아래는 "왜"에 대한 구체적인 답이다.

이 구조가 잘 맞지 않는다고 말하는 세 출처를, 그대로 인용한다

"모든 에이전트가 같은 컨텍스트를 공유해야 하거나 에이전트 간 의존성이 많은 도메인은 멀티에이전트에 안 맞는다."^[1] "나는 멀티에이전트 협업의 결과 품질을 예측하기 어렵다고 본다."^[2] "순차적 작업·동일 파일 편집·의존성 많은 작업엔 단일 세션/Subagent가 더 낫다."^[3] 이 설계는 셋 다 해당한다 — 04절이 "4개 에이전트가 공통으로 같은 문서를 읽는다"고 규정한 것 자체가 컨텍스트 공유 요구의 자백이고, Implementer→Reviewer→Verifier는 강한 순차 의존이다.

그럼에도 이 구조를 쓰는 이유는 처리량이나 병렬성이 아니다. Reviewer/Verifier가 Implementer의 추론 경로를 상속받지 않고 독립적으로 판단하게 만드는 것, 그리고 컨텍스트 격리 자체를 하나의 게이트로 쓰는 것이 목적이다 — 위 세 인용 중 어느 것도 이 목적을 반박하지 않는다. [1]이 경고하는 워크로드는 병렬 리서치용으로 최적화된 것이라, 이 설계와는 애초에 비교 대상이 다르다.

논증 사슬 — 이 구조가 서는 순서

위 반론들을 감수하고도 이 형태가 되는 이유를 한 줄씩 끊어서 적는다. 각 단계에 **근거의 종류**를 같이 붙인다 — 외부 근거인지, 스스로 선 논증인지, 아직 검증되지 않은 가정인지 구분하지 않으면 사슬이 어디서 끊기는지 보이지 않는다.

1. LLM은 확률적 기계다. 같은 맥락 안에서 이뤄지는 자기검토는 그 맥락이 만들어낸 오류를 구조적으로 놓친다 — 틀린 전제를 공유한 채 결론만 다시 읽기 때문이다. [자체 논증]
2. 따라서 판정자는 구현자의 추론 경로를 물려받지 않아야 한다. 여기서 필요한 것은 다른 **프롬프트**가 아니라 다른 **컨텍스트**다. 같은 세션 안에서 단계만 나누는 방식은 대화 이력을 그대로 공유하므로 이 조건을 만족하지 못한다. [자체 논증]

3. Claude Code에서 컨텍스트 경계는 곧 프로세스 경계다. 그래서 독립 판정을 강제하려면 역할마다 별도 프로세스여야 하고, 세션 간 전달은 순수 텍스트뿐 — 보내는 쪽의 이력이나 파일은 따라가지 않는다^[25]. 격리가 도구 차원에서 보장된다는 뜻이다. [도구 사실 · 문서 확인]
4. 별도 프로세스는 조상 문서를 매번 다시 읽는 고정비를 낳는다. 이 비용은 프리픽스 캐싱으로 상쇄된다 — 비용 41~80% 절감, TTFT 13~31% 개선^[18], 인프라 레벨 선례도 있다^{[19][20]}. 성립 조건은 조상 프리픽스가 정적이라는 것이다(04절). [외부 근거 · 조건부]
5. 그렇다면 "긴 세션 하나에 다 시키면 되지 않나"가 남는다. 임상 규모 워크로드 실험에서 단일 지속 에이전트는 작업량이 늘자 정확도가 73.1%→16.6%로 붕괴한 반면, 오케스트레이션된 다중 에이전트는 90.6%→65.3%로 버텼고($p<0.01$) 토큰은 최대 65배 적었다^[21]. 그래서 이 설계는 상시 대기하는 장수 세션이 아니라 **태스크마다 스폰하고 끝나면 회수하는** 형태를 택한다(04절). [외부 근거]
이 근거가 닿는 범위: 도메인은 임상 워크로드이고 측정한 것은 정확도·토큰이다. 따라서 "역할을 나누면 판단이 좋아진다"까지 끌어 쓰지 않고, **장수 세션보다 태스크 단위 스폰·회수가 낫다**는 단계에만 쓴다. 판단 독립성 자체의 값은 8번의 실측이 따로 책임진다.
6. 판정자를 여럿 두면 판정이 갈린다. 갈린 판정을 수렴시킬 단일 지점이 없으면 AC 루프가 끝나지 않는다. 그래서 Reviewer는 의견을 내고 **Verifier 하나가 최종 게이트**가 된다 — 격리는 판단을 독립시키기 위한 것이지 결정을 분산시키기 위한 것이 아니다. [자체 논증]
7. 그 게이트가 무엇을 통과시킬지는 일반 절차로 정할 수 없다. 임의의 명세가 충족됐는지 판정하는 보편적 방법은 존재하지 않으므로(결정문제), 판정기는 프로젝트마다 유한하게 쓰는 수밖에 없다 — 그것이 AC와 DoD, 그리고 테스트다. 미들웨어는 그 결과의 Exit Code만 심사한다(07절). [원리적 제약 · 자체 적용]
8. 남는 공백은 그 유한한 판정기의 **내용**이다 — Reviewer의 pass/fail 기준과 Verifier·AC 판정 기준(09·11절). 그래서 먼저 채점 기준을 요구하지 않는 walking skeleton으로 실행 기반부터 세우고(12절), 사슬 전체는 아래 중단 기준으로 검사한다. [미해결 — 측정 대상]

사슬이 끊기는 지점은 둘이다. 4번의 캐싱이 성립하지 않으면 3번의 비용이 정당화되지 않고, 8번의 실측이 이득을 보이지 않으면 1~2번의 전제가 틀린 것이다. 둘 다 아래 중단 기준과 12절 실측으로 직접 확인한다.

반증 가능한 중단 기준: Implementer 단독 + self-review 대비, 4-agent 구조가 AC 재작업률·회귀 발생률을 유의미하게 낮추지 못하면 Reviewer/Verifier를 별도 프로세스가 아니라 Implementer 세션 내 단계(step)로 접는다. 이 판단은 12절의 walking skeleton 이후 첫 실측으로 검증한다.

01 계층 구조 — 프로젝트 / 마일스톤 / 태스크 / AC

backlog.md^[27]의 실제 구조를 그대로 따른다: 마일스톤은 **중첩되지 않는 평평한 목록**이고(`milestone add/rename/remove/archive`), 태스크는 마일스톤에 속하지 않고 독립적으로 존재할 수도 있으며, 대신 **태스크가 다른 태스크를 부모로 가질 수 있다**(`parentTaskId`, 서브태스크). 깊이가 필요한 곳은 마일스톤이 아니라 태스크 쪽이다.

계층이 코드의 무엇에 대응하는가

backlog.md의 계층은 그 자체로는 추상적이다. 이 설계에서는 각 층을 코드의 단위에 **고정해서** 쓴다 — 이 대응이 격리 규칙과 커밋 단위를 동시에 결정한다.

- **태스크 = 파일 하나.** 태스크 카드 하나가 소스 파일 하나에 대응한다.
- **AC = 함수 하나.** PM은 태스크 아래를 함수 단위까지 쪼개 AC로 적는다. 따라서 한 태스크의 AC 개수는 그 파일에서 건드릴 함수의 개수다.
- **커밋 = 함수 하나(=AC 하나).** Implementer는 AC를 하나 끝낼 때마다 커밋한다. 커밋이 함수 단위이므로 git-format의 post-commit 토큰 계량도 함수 단위로 붙는다.
- **에이전트 배정 = 파일 하나(=태스크 하나).** 04-0의 *1태스크 1에이전트 1파일 1주제* 규칙이 여기서 나온다. 파일이 격리 단위이므로 워크트리를 분리하면 두 에이전트가 같은 파일을 만질 일이 구조적으로 없다.

이 대응의 부수 효과: 태스크당 최소 커밋 수를 배정 전에 알 수 있다 — 카드의 AC 개수가 곧 그 값이다. 실제 커밋 수는 여기에 자가 치유 재시도가 곱해진 만큼 늘어나며, 그 배수는 아직 실측되지 않았다(05절 커밋 정리 방식 미정의 근거).

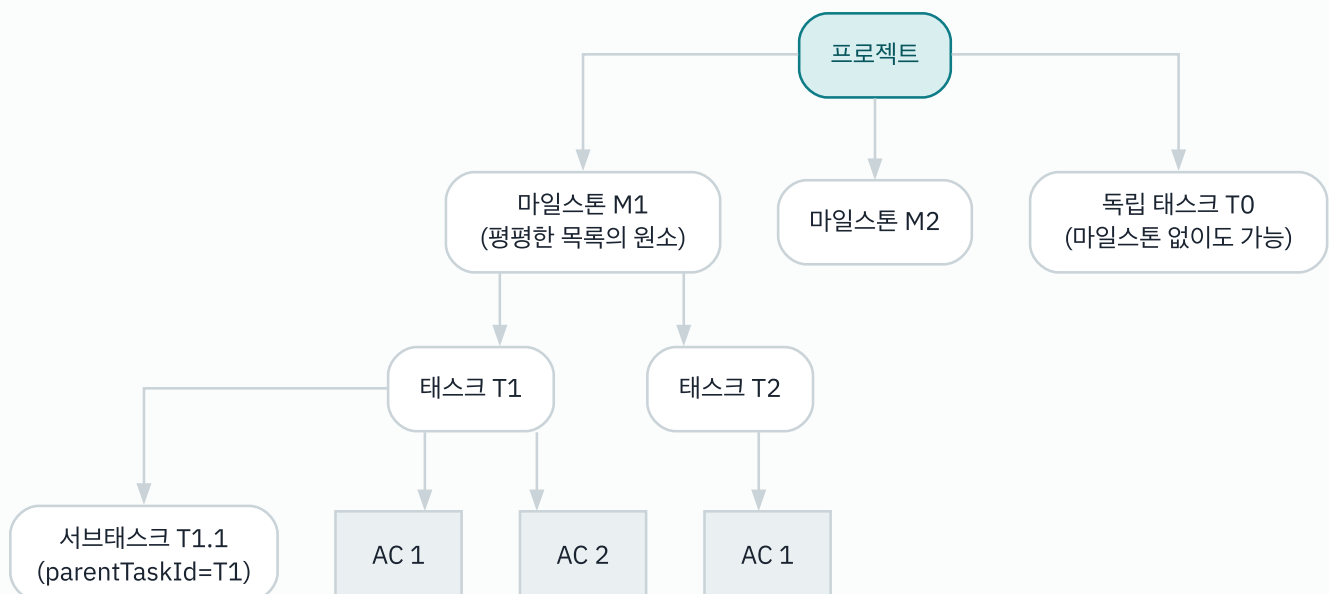


FIG.1 - BACKLOG.MD 상의 실제 계층. 마일스톤은 평평하고, 중첩은 태스크의 PARENTTASKID로만 표현된다

상태 값은 08절 참고 — backlog.md의 실제 기본값 세 가지만 쓴다.

02 Phase A - 저장소 부트스트랩 (1회, role 분리 전)

backlog 명령을 쓰려면 CLI와 init이 먼저 있어야 하고, 백로깅 절차 자체를 강제하는 claude-rails도 백로그 생성 이전에 있어야 한다 — "백로그 먼저, 도구는 나중"은 성립하지 않는다.

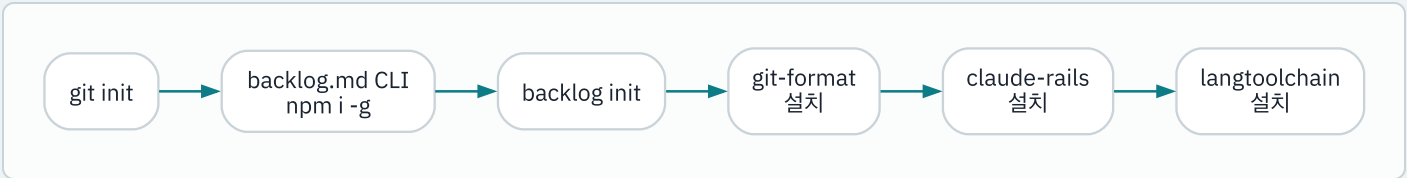


FIG.2 - 이 순서를 벗어나면 다음 단계 명령 자체가 실행 불가

컴포넌트	역할	왜 이 순서 · 이 도구인가
git init	버전관리 저장소 생성	이후 모든 도구(브랜치, 커밋훅, worktree)가 .git 존재를 전제로 동작
backlog.md CLI	마일스톤 · 태스크 · AC 계층과 상태(To Do/In Progress/Done)를 규정하는 명령 제공	CLI 자체가 없으면 draft/promote 등 이후 모든 백로깅 명령이 존재하지 않음
backlog init	이 저장소 전용 backlog 데이터 폴더 생성	CLI가 전역에 있어도, 프로젝트별 초기화 없인 draft를 담을 곳이 없음
git-format	pre-commit/commit-msg/post-commit 훅으로 커밋 형식 강제	AI 에이전트가 남기는 방대한 커밋을, 사람도 LLM도 git log 한 번으로 신뢰성 있게 읽게 하기 위함
claude-rails	Claude Code 훅으로 브리핑→draft→promote, task view 필독, In Progress+브랜치 확인, 커밋 전 테스트를 강제	CLAUDE.md에 적어두는 약속만으론 프롬프트 인젝션·컨텍스트 압축으로 깨질 수 있어, 기계적으로 강제
langtoolchain	Node/Java/Python/Rust/Go 런타임 설치	Implementer가 실제 코드를 짜기 직전에만 필요 — 순서상 유일하게 뒤로 미뤄도 되는 항목

03 Phase B – 백로깅 (프로젝트 진행 중 반복 가능)

최초 1회만이 아니라, 진행 중 새 마일스톤/백로그가 필요해지면 언제든지 재진입한다. 단일 Claude 세션이며, 아직 4-agent 분리 전이다.

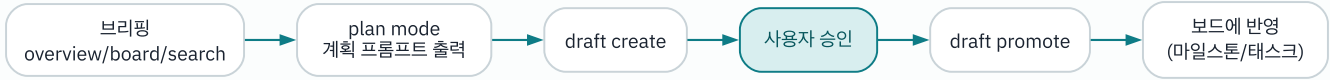


FIG.3 - PHASE C 도중에도 이 루프로 되돌아올 수 있음 (FIG.4의 점선 참고)

재진입은 임의로 일어나지 않고, 아래 조건에서만 트리거된다:

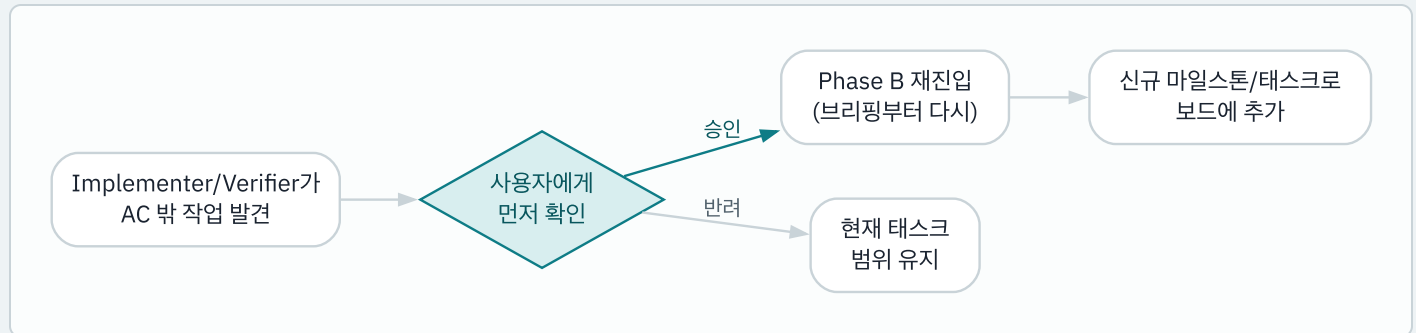


FIG.3-1 - 브리핑을 다시 하는 이유는 이미 있는 마일스톤/태스크와의 중복 생성 방지. 진행 중인 다른 태스크(PHASE C)는 브랜치/WORKTREE가 분리돼 있어 이 재진입과 서로 간섭하지 않는다 - 보드에 새 노드가 추가될 뿐이다.

왜 마일스톤 · 태스크 단위로 쪼개서 backlog.md에 남기는가

- 에이전트가 나보다 똑똑할 때 - 태스크 단위로 남은 계획·결정·커밋 이력을 그대로 따라가며 그 판단 과정을 배울 수 있다.
- 에이전트가 나보다 멍청할 때 - 통짜 결과물이 아니라 작은 단위로 쪼개져 있어야, 어디서부터 잘못됐는지 그 지점을 정확히 짚어낼 수 있다.
- 다른 사람이 끼어들 수 있게 - 하나의 태스크(단위 업무)가 곧 하나의 리뷰 체크포인트가 되어, 본인이 아닌 다른 사람도 그 지점에서 피드백을 주거나 도움을 줄 수 있다.

draft → 승인 → promote 게이트(fig.3)가 이 체크포인트가 실제로 걸리는 지점이다.

04 Phase C - 태스크 실행 (태스크마다 4-agent 동적 스폰)

Phase B에서 나온 태스크가 Ready 되면, 그 태스크 전용으로 4개 에이전트가 그때 생성되고 끝나면 회수된다 - 상시 대기 중인 4개가 아니다. Andrew Ng의 Multi-Agent Collaboration 패턴("서로 다른 역할을 맡은 여러 LLM 인스턴스가 문제를 푸는 것")을 태스크 단위로 인스턴스화한 것이다 (10절).

04-0. 에이전트 할당 전 - 실행 순서 결정

Phase B가 한 번에 여러 태스크/마일스톤을 Ready로 만들 수 있으므로, 4-agent 생성으로 넘어가기 전에 먼저 실행 순서를 정한다. 판단 신호는 두 가지다: ① 선언된 의존성과 ② 파일/모듈 결합도(정적분석)

로 측정 — 명시적 의존성이 없어도 같은 파일을 건드리면 사실상 결합돼 있다고 본다). 둘 중 하나라도 있으면 그 태스크들은 순차 체인으로 묶고, 둘 다 없는 노드끼리만 병렬로 배정한다.

신호가 하나 더 있다 — **사람이 선점한 파일**. 사람이 태스크를 직접 In Progress로 옮기면 그 태스크에 대응하는 파일에 락이 걸리고, 할당자는 그 파일을 건드리는 태스크에 에이전트를 배정하지 않는다. 태스크가 파일 하나에 대응하므로(01절) 락의 단위도 파일 하나로 떨어지고, 상태 값 자체가 락이 되어 별도 잠금 파일이나 프로토콜이 필요 없다. 사람이 In Progress에서 내려놓는 순간 락도 풀린다.

이 락이 막는 것은 에이전트끼리의 충돌이 아니라 **사람과 에이전트의 충돌**이다. 에이전트끼리는 워크트리 분리로 이미 물리적으로 갈라져 있지만, 사람은 메인 작업 트리에서 같은 파일을 열고 있을 수 있다.

①은 backlog.md^[27]가 이미 제공하는 기능이다 — `backlog task edit 7 --dep task-1 --dep task-2`로 의존성을 선언하면, "완료되지 않았고 선언한 의존성이 전부 완료된 태스크"인지를 뜻하는 `isReady`를 backlog.md 스스로 계산해준다. 즉 06절 Board Watcher는 의존성 그래프를 직접 만들 필요 없이 이 필드를 그대로 읽으면 된다. 실제 출력 형태:

Dependency Graph:

Depends on (1 direct, 2 total):

└─ BACK-2 - Parser rewrite [In Progress]
 └─ BACK-1 - Token schema [completed]

Dependents (2 direct, 2 total):

└─ BACK-8 - Migration guide [To Do]
└─ BACK-9 - Release checklist [To Do]

FIG.4-0A — BACKLOG TASK <ID>의 실제 출력(BACKLOG.MD CLI-INSTRUCTIONS.MD^[27]). "DEPENDS ON"이 04-0의 순차 체인, "DEPENDENTS"가 그 체인이 끝나야 풀리는 하류 태스크다.

"순차 다음 병렬"은 이 문서에서 두 층위로 따로 적용된다 — 혼동하지 않도록 구분해둔다. **태스크 사이**에서는 이 절의 규칙대로 의존·결합이 있는 태스크만 순차로 묶고, 나머지는 즉시 병렬로 돌린다(워커풀을 놀리지 않기 위해 완전히 다 끝날 때까지 기다리지 않는다). **한 태스크 안에서**는 Implementer의 AC 구현 자체는 같은 브랜치·같은 파일이라 본질적으로 순차이지만, 커밋 하나가 나올 때마다 Reviewer와 Verifier·DoD는 서로를 기다리지 않고 동시에 판단한다(fig.5) — "구현은 순차, 그 결과에 대한 판단은 병렬"이라는 게 이 설계의 실제 형태다.



FIG.4-0 — 06절 REQUEST QUEUE가 이 그래프를 입력으로 받아, 순차 체인은 선행순서를 지켜 큐에 넣고 독립 노드는 SEMAPHORE 용량 안에서 한 번에 여러 개를 ADMIT한다

지켜야 할 제약은 "체인을 먼저 보낸다" 자체가 아니라 워커풀을 **work-conserving**하게 유지하는 것이다 — 순차 체인의 admission이 독립 노드가 채울 수 있는 빈 슬롯을 막아서는 안 된다. 그 제약만 지키면 이

방식은 처리량 최적성을 잃지 않는다. 순차 체인의 길이는 전체 실행 시간의 하한이 되며(Gene Amdahl의 Amdahl's Law, 1967), 이 하한은 병렬화로 없앨 수 없다 — 다만 이 프로젝트처럼 백로그가 고정돼 있지 않고 계속 재생성되는 구조에서는 John Gustafson의 Gustafson's Law가 더 실질적인 모델이다: 처리 용량이 늘면 처리량도 거의 선형으로 늘 수 있다. 병렬로 동시에 도는 에이전트 수는 무한정 늘리지 않는다 — 같은 코드베이스를 동시에 건드리는 에이전트가 많아질수록 서로 동기화되지 않은 채 벌어질 위험이 조합적으로 커진다(Fred Brooks의 *맨먼스 미션*(1975)이 사람에게 대해 말한 것과 같은 형태의 경고이며, 에이전트에도 그대로 적용되는지는 아직 동료심사로 증명되지 않았다).

이 형태는 새로 만드는 개념이 아니라 여러 곳에서 독립적으로 검증된 표준 패턴이다: HEFT(IEEE, critical-path 우선 랭킹), DynTaskMAS(동료심사 저널 *Electronics*, DOI:10.3390/electronics15112475, LLM 멀티에이전트용 동적 태스크그래프), Airflow의 dynamic task mapping, Argo Workflows, Netflix Conductor 모두 "DAG 생성 → 위상정렬 → 병렬 가능한 노드는 동시 실행"과 같은 형태다. 파일 결합도까지 반영하는 구체적 선례는 10절 참고.

스케줄 계측 — 순차 그래프의 길이와 폭을 센다

위 규칙은 순서만 정할 뿐, **지금 이 백로그가 얼마나 병렬화될 수 있는 구조인가**를 말해주지 않는다. 그래서 Ready 집합을 큐에 넣기 전에 두 수치를 계산해 함께 기록한다 — 의존 간선(--dep)과 결합 간선(같은 파일/모듈)을 합친 DAG를 위상정렬한 뒤,

- **길이 — 임계 경로(critical path).** 가장 긴 순차 체인의 노드 수. 이 값이 전체 소요의 하한이고, 아무리 용량을 늘려도 줄지 않는다 (Amdahl). HEFT의 critical-path 랭킹과 같은 계열이되, 여기엔 노드별 실행시간 예측이 없으므로 시간이 아니라 노드 수로 잰다.
- **폭 — 동시 배정 가능한 태스크 수.** 서로 의존·결합이 없는 노드를 한 번에 몇 개까지 띄울 수 있는가. 위상 레벨별 노드 수의 최대값으로 근사하고, 정확한 값이 필요하면 최대 antichain(Dilworth 정리 — 이분 매칭으로 계산)으로 낸다.

폭은 곧 **이 시점에 의미 있는 동시성의 상한**이다. 폭보다 큰 Semaphore 용량은 채울 일감이 없어 낭비되고, 폭보다 작은 용량은 독립 노드가 놀게 만들어 앞 문단의 work-conserving 제약을 스스로 어긴다. 따라서 실제 동시성은 $\min(\text{그래프 폭}, \text{자원 상한})$ 으로 잡는다 — 06절이 자원 쪽(peak RSS·API 한도)만 근거로 삼던 것을 그래프 쪽 상한과 함께 본다.

두 수치는 Logger가 매 스케줄 라운드마다 남긴다: Ready 집합 크기 · 임계 경로 길이 · 폭 · 실제 동시 가동 수. 이것이 사후에 "이 백로그에서 병렬화가 실제로 이득이었는가"를 계산할 수 있는 유일한 자료이고, 백로그가 계속 재생성되는 이 구조에서 Gustafson 쪽 가정(용량을 늘리면 처리량도 는다)이 실제로 성립하는지 확인하는 방법이기도 하다.

반증 조건: 실측된 폭이 거의 항상 1~2에 머문다면, 이 백로그는 애초에 병렬 배정할 구조가 아니므로 워커 풀·Semaphore·결합도 분석 전체가 과설계다 — 그때는 순차 실행 하나로 접는다.

태스크 Ready
(위 순서대로)

4-agent 동적 생성
Implementer/Reviewer/Verifier/Logger

AC 루프
Implementer 커밋마다 Reviewer+Verifier·DoD 병렬 판단

전체 AC 종료 →
Verifier·AC 최종 판정

check-ac + check-dod
통과 시 Done + final-summary

신규 마일스톤/백로그 필요 발견

Phase B 재진입

FIG.4 - 점선은 실행 중 재백로깅이 트리거되는 유일한 경로. AC(개별 완료 조건)와 DOD(공통 완료 기준)는 BACKLOG.MD가 이미 --CHECK-AC/--CHECK-DOD로 함께 제공하는 구분이며, C5로 넘어가려면 둘 다 통과해야 한다 - 07절 VERIFIER의 두 갈래(DOD/AC)는 이 기존 구분을 그대로 따른 것일 뿐 새로 만든 개념이 아니다.

태스크 하나 안에서 실제로 오가는 순서:

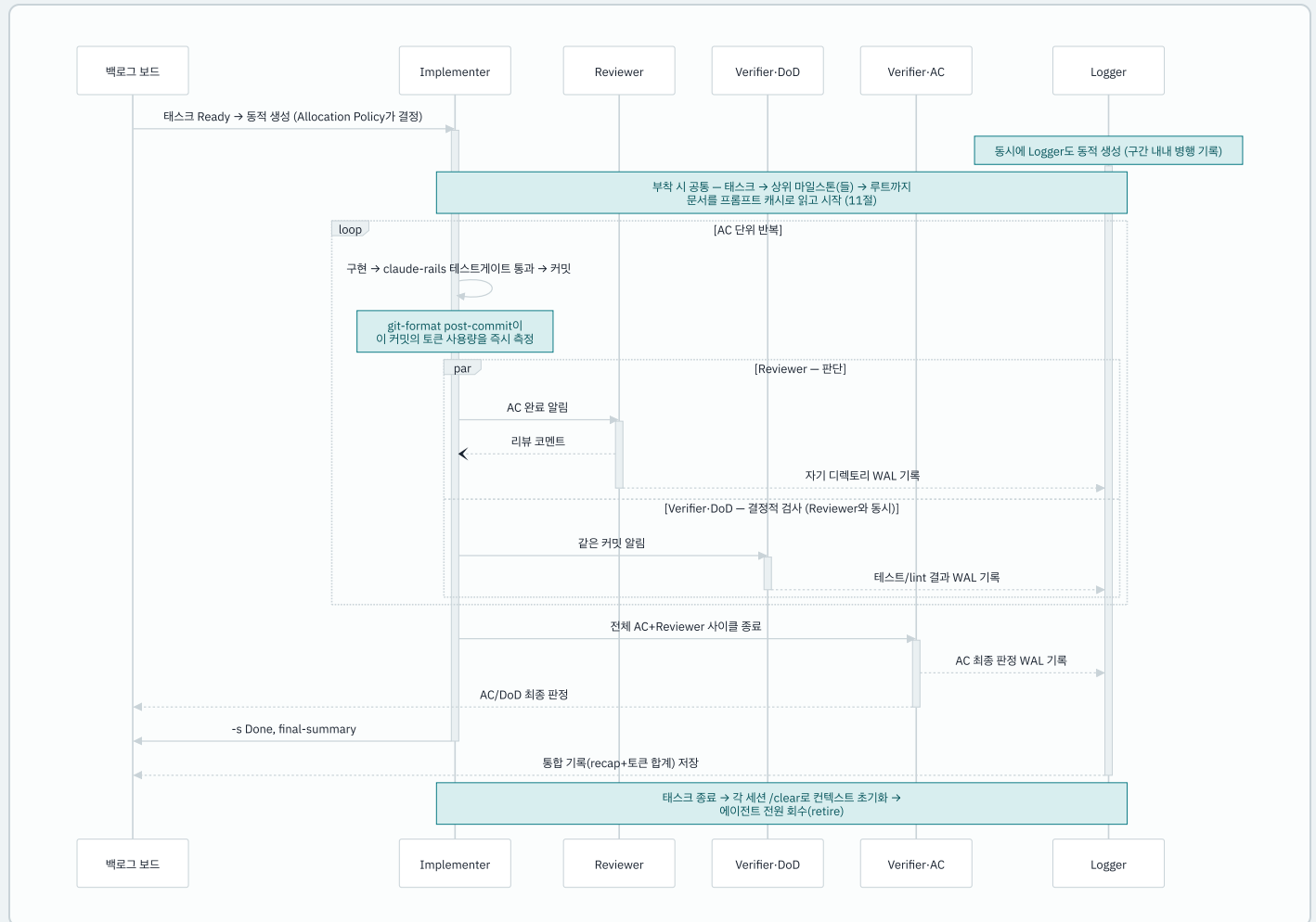


FIG.5 - PAR 블록이 REVIEWER와 VERIFIER·DOD의 동시 활성화 구간. VERIFIER·AC만 전체 사이클 종료를 기다린다

같은 순간에 맞물리는 것들 - Implementer가 AC를 커밋하는 순간

위 루프의 커밋 한 번을 확대하면 이렇다. ① claude-rails의 pre-commit/post-commit 훅이 테스트 게이트와 커밋 형식을 강제한다 - Implementer가 유일한 writer라는 07절의 권한 구분은 선언이 아니라 이 훅이 실제로 강제하는 것이다. ② 같은 순간 git-format의 post-commit이 그 커밋의 토큰 사용량을 측정해 트레일러로 남긴다(06절) - Logger는 이 값을 나중에 물어보는 게 아니라 커밋 자체에서 읽는다. ③ Reviewer와 Verifier·DoD가 동시에(par) 깨어나 그 AC를 각자의 기준으로 본다 - 순차 릴레이가 아니라 병렬인 이유는 06절에서 인용한 work-conserving 원칙 때문이다: Verifier·DoD가 Reviewer의 판단을 기다릴 이유가 없는데도 기다리게 하면 위커풀의 빈 슬롯을 스스로 놀리는 셈이다. ④ 이 둘이 각자 독립적으로 판단할 수 있는 이유는 새로 문서를 읽지 않기 때문이다 - 부착 시 캐싱된 같은 조상 문서 프리픽스를 그대로 재사용한다(11절②). ⑤ Logger는 이 전 과정을 관찰만 하며 WAL에 쌓는다. 즉 이 한 순간에 claude-rails(제어), git-format(계측), Allocation Policy(누가 언제 깨는지), 프롬프트 캐싱(왜 병렬이

저렴한지)이 전부 동시에 작동한다 — 이 문서의 여러 절에 흩어져 있던 결정들이 실은 이 한 장면을 만들기 위한 것이다.

부착 시 컨텍스트 로딩 • 회수 시 컨텍스트 폐기

- **부착(생성) 시** — 각 에이전트는 자기 역할과 무관하게, 해당 태스크 문서 하나만 보는 게 아니라 그 태스크가 속한 상위 마일스톤(들)을 거쳐 프로젝트 루트까지 이어지는 문서를 전부 읽고 작업을 시작한다(01절 계층 트리를 아래→위로 훑는 것과 동일). 이 조상 문서 프리픽스는 프롬프트 캐싱의 대상이므로 **타임스탬프·세션ID 같은 가변값을 섞지 않고 정적으로 유지해야 한다**(11절).
- **회수(retire) 시** — 태스크가 끝나면 세션에 /clear를 실행해 컨텍스트를 비운 뒤 회수한다. 다음 태스크에 새로 붙는 에이전트는 이전 태스크의 대화 이력을 이어받지 않고 위 문서 재로딩으로만 컨텍스트를 구성한다 — Logger의 WAL 기록이 그 컨텍스트를 잃지 않게 하는 유일한 지속 장치다.

05 에이전트 부착 지점 — 태스크 전용이 아니라 계층 어디든

4-agent는 태스크에만 고정 부착되지 않는다. 마일스톤을 포함해 계층의 어느 노드에도 붙을 수 있고, 노드마다 붙는 개수도 4로 고정이지 아니라 2~4 등으로 달라질 수 있다.

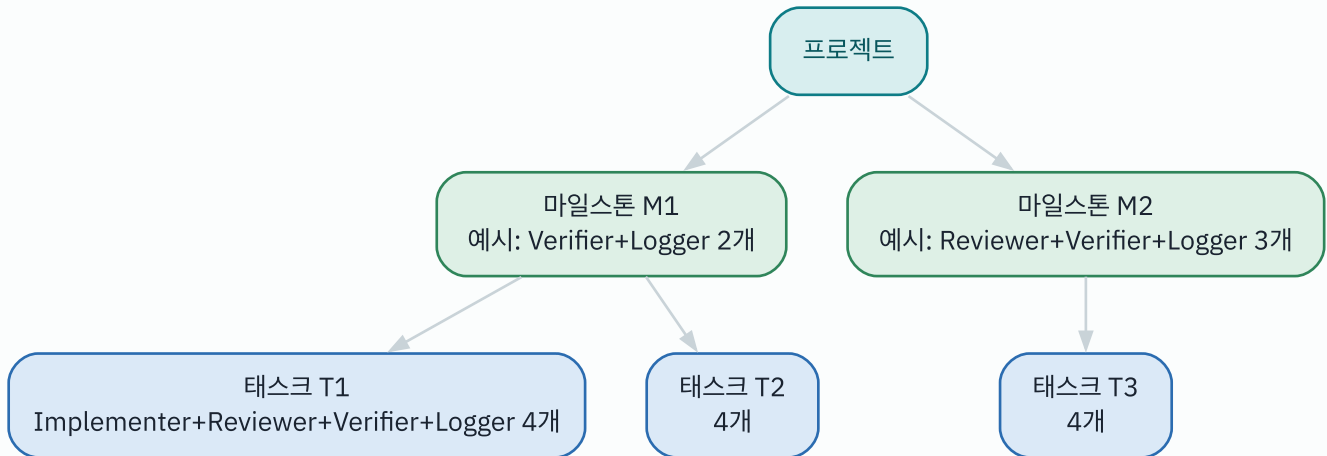


FIG. 6 — 예시 • 미확정 마일스톤 단위 에이전트가 실제로 뭘 검증·기록하는지는 아직 정의되지 않음

마일스톤 노드 에이전트는 매니저가 아니라 집계자다

fig. 6의 마일스톤 노드 에이전트는 자식 태스크에 작업을 배분하지 않는다. 이미 Done인 자식들의 산출물을 읽어 통합 검증·기록만 하는 **집계자(aggregator)**다. 작업 배분 권한은 오직 06절 Allocation Policy에만 있다 — 11절에서 다루는 "계층-위임(매니저) 오버헤드"에 대한 경계는 이 노드에 적용되지 않는다.

"어느 노드에 몇 개, 어떤 역할을 언제 붙일지"는 이 문서가 정하지 않는다 — 06절 오케스트레이터가 외부에서 주입하는 정책의 몫이다.

06 에이전트 투입 오케스트레이터 (JVM)

"언제 · 얼마나 · 어떤 역할을 · 어떤 모델로" 스폰할지를 오케스트레이터 내부에 하드코딩하지 않고, 교체 가능한 정책으로 외부 주입한다.

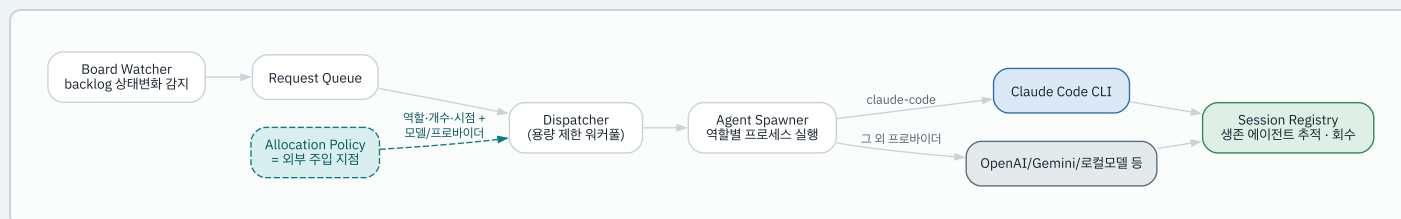


FIG. 7 - POLICY는 "노드+현재상태 → (역할, 개수, 시점, 모델/프로바이더)"를 반환하는 인터페이스 하나로 갈라지운다

확정된 설계 결정

- **독립 프로세스** — 제품군이 다양해 역할마다 다른 설정·권한·작업 디렉토리가 필요하므로, Subagent나 Agent Teams(세션 내 위임)는 쓰지 않는다. 4개 역할 전부 완전히 독립된 프로세스로 띄운다(fig. 7의 Agent Spawner). 기본은 `claude CLI` 프로세스지만, 역할 하나당 다른 LLM 프로바이더/CLI로 교체할 수 있다(예: Verifier만 다른 프로바이더로 교차검증 — OpenWorker의 멀티 프로바이더 지원에서 가져온 아이디어). 이 방식이 토큰 측정도 더 쉽게 만든다 — 세션 내부 위임이면 부모·자식 사용량이 한 응답에 섞이지만, 독립 프로세스면 각 프로세스 종료 시 자기 몫의 사용량만 딱 떨어지게 나온다.
- **언어·동시성** — Java 25(JVM), `virtual thread` + `Semaphore`로 동시성을 처리한다. 이 설계의 동시성은 한 자릿 수~십수 개의 OS 서브프로세스 규모라 동시성 모델 자체가 성능을 좌우하지 않는다 — `virtual thread`의 실익은 `Process.waitFor()`를 그냥 블로킹으로 써도 되어 비동기 콜백 지옥이 사라지는 코드 단순성이다. 즉시 스폰과 비동기 큐 중에서는 이분법 대신 **용량 제한 워커풀**로 절충한다 — 평상시엔 즉시 디스패치처럼 동작하다 트리거가 몰릴 때만 자연스럽게 큐잉된다. AC 루프 안의 Reviewer처럼 지연에 민감한 경로는 별도 전용 슬롯으로 떼어둔다. `Semaphore` 상한은 벤치마크가 아니라 (a) `claude` 프로세스 1개의 peak RSS와 (b) 본인 API 플랜의 분당 요청 한도로 정한다 — 이 둘을 재는 게 walking skeleton 다음 실측 항목이다. 여기에 (c) 04-0에서 잔 **그래프 폭**이 더해져, 실제 동시성은 $\min(\text{그래프 폭}, \text{자원 상한})$ 이 된다 — 자원이 남아도 띄울 독립 노드가 없으면 용량은 의미가 없다.
- **정책 주입** — `AllocationPolicy` 인터페이스 하나 + YAML 기반 기본 구현 + `-Dpolicy.class=` 시스템 프로퍼티로 교체하는 탈출구. Kubernetes Scheduler Framework처럼 코어는 그대로 두고 정해진 확장 지점만 갈라끼우는 형태를 참고했다(10절) — 나중에 SPI로 바뀌도 호출부는 바뀌지 않는다.
- **지연/비용 마진** — 태스크마다 개별 버퍼를 두지 않고 오케스트레이터 전체 차원의 공유 버퍼 하나로 관리한다 (Critical Chain Project Management의 자원 제약 인지 방식을 차용 — LLM 에이전트에 적용한 선행 연구는 없

고, 고전 PM 이론에서 직접 가져온 확장이다). Semaphore로 동시 작업량을 제한하는 것 자체는 큐잉이론의 Little's Law(리드타임 = $WIP \div$ 처리량)로도 독립적으로 뒷받침된다.

완료 시점의 기록 – 커밋을 읽어 태스크 카드에 남긴다

태스크를 Done으로 옮기는 시점(`backlog task edit <ID> -s Done --final-summary ...`)에, 그 태스크 브랜치의 커밋들을 읽어 요약을 태스크 카드에 기록한다. 커밋이 함수 단위이므로(01절) 함수별 작업 내역과 토큰 집계도 이 시점에 카드 안으로 들어온다.

이 한 단계가 뒤에 나올 선택 하나를 풀어준다 – **이력을 나중에 어떻게 정리하든 기록은 남는다**. 커밋을 합치면 `git log`에서는 함수별 흔적이 사라지지만, 그 내용은 이미 태스크 카드(그리고 그 카드를 담은 커밋)에 적혀 있다. 따라서 스쿼시 여부는 기록 보존의 문제가 아니라 이력 가독성의 문제로만 남는다(아키텍처 문서 5절).

토큰 소모량 측정 – 방향은 확정, 정확도는 미정

- LLM은 생성 중 자기가 몇 토큰 썼는지 아는 내장 카운터가 없다 – 토큰 수는 모델이 아니라 API가 매 응답에 실어 돌려주는 메타데이터다. 그래서 Logger가 Implementer/Reviewer/Verifier에게 "얼마나 썼어?"라고 물어보는 설계는 성립하지 않는다.
- `~/githubs/git-format`의 `post-commit` 혹은(`measure_claude_code_token_usage()`)이 이미 프로덕션에서 쓰는 실측 방식을 갖고 있다: Claude Code 세션 트랜스크립트(`~/claude/projects/<slug>/<session-id>.jsonl`)에서 `assistant` 메시지의 `message.usage`(API 응답을 그대로 기록한 값)를 합산하고 `tool_use` 블록 개수를 센다. 세션 하나에서 커밋이 여러 번 나오는 걸 감안해 "이미 처리한 트랜스크립트 줄 수"를 커서 파일에 저장해두고 그 이후 구간만 델타로 집계한다. 측정 실패 시에는 조용히 틀린 값 대신 `unavailable (no-session-id / jq-not-installed / transcript-not-found / transcript-parse-failed)` 같은 사유를 남긴다 – 이 실패-투명성 패턴은 그대로 가져다 쓸 만하다.
- `git-format` 저장소 스스로도 이 값이 "그 커밋에 정확히 필요했던 토큰"이 아니라 "직전 커밋 이후 세션에서 소비된 토큰"의 근사치라고 밝히고 있다 – 무관한 대화가 섞이면 과대평가될 수 있다. 그래서 이 측정 방식은 **채택하되 확정으로 표시하지 않는다**.
- 이 델타 커서는 Implementer처럼 한 세션이 여러 커밋에 걸쳐 계속 살아있는 경우엔 그대로 재사용한다. Reviewer/Verifier처럼 프로세스가 짧게 한 번만 도는 경우엔 커서 없이 그 트랜스크립트 전체를 한 번만 합산하면 된다 – 헤드리스 실행(`claude -p ...`) 여부와 무관하게 트랜스크립트는 동일하게 남는다.

남은 미해결 항목은 09절에 모아뒀다: 보드 상태 감지 방식(CLI 폴링 vs WatchService), 다중 프로바이더 통합에 별도 라이브러리가 필요한지 여부.

역할	코드 / BACKLOG.MD 쓰기	트리거 시점	산출물	비정상 종료 시	책임 범위
IMPLEMENTER	O — 유일한 writer	태스크 Ready 시 생성	커밋, backlog 상태 변경	재스폰 안 함 — 부분 커밋 상태 라 사람이 판단	확정
REVIEWER	X — 읽기 전 용	AC 완료마다(순 차 호출, 최대 N 회 왕복 후 사람 호출 — 무한 루 프 방지)	자기 디렉토리 WAL recap + 판정 스키마 {""}ac_id, verdict, evidence, blocking{""}	N회까지 재 스폰, 초과 시 태스크 blocked	출력 스키 마는 확정. 무엇을 pass/fail 로 볼지(채 점 기준)는 PKM 완성 후
VERIFIER • DoD	X — 읽기 전 용	AC 완료마다, Reviewer와 동 시(병렬) — Reviewer의 판 단을 기다리지 않 음(fig.5 par)	결정적 체크(테스 트/lint 종료코드 등) 결과	N회까지 재 스폰, 초과 시 태스크 blocked	확정 — PKM 불필 요, 오늘 구 현 가능
VERIFIER • AC	X — 읽기 전 용	전체 AC+Reviewer 사이클 종료 후 1 회	AC 판정(같은 스키마)	N회까지 재 스폰, 초과 시 태스크 blocked	PKM 완성 후 명세
LOGGER	X — 관찰만	태스크 시작~종 료 전 구간 상시	Reviewer·Verifier WAL 취합 + Implementer 커밋 이력(커밋 자체가 세 번째 기록 소스)	해당 없음 — 관찰자라 재스폰 불 필요	토큰 계량 은 확정 (06절). recap 같 은 정성적 기록의 구 현 위치는 TBD

08 백로그 상태 범례

backlog.md^[27]가 실제로 쓰는 상태 값은 세 가지뿐이다 — 모든 CLI 예시가 이 표기로 통일돼 있다. Phase B/C의 "보드"는 이 세 값을 그대로 쓴다.

● To Do

● In Progress

● Done

"blocked"는 상태 값이 아니라 backlog.md가 의존성으로부터 계산하는 파생 속성(isBlocked, 04-0절의 isReady와 짝)이고, "archive"는 상태가 아니라 milestone archive류의 별도 동작이다 — 둘 다 이전 판(v19 이전)에서 상태 값으로 잘못 나열돼 있었다.

상태를 몇 개까지 둘 것인가 — 상반된 근거를 함께 본다

상태를 적게 두는 쪽 근거: David J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*^[31](2010)의 핵심 원칙은 "지금 하는 방식을 있는 그대로 시각화하고, 병목이 실제로 증명될 때만 구조를 늘려간다"는 것이다. backlog.md의 3상태 기본값은 이 원칙의 임의 단순화가 아니라 정석적인 사례다. 다만 정직하게 밝히면, "상태 개수와 인지부하"를 직접 실험으로 재는 연구는 찾지 못했다 — 이 근거는 검증된 실증 결과가 아니라 확립된 방법론 원칙이다.

상태를 늘리는 쪽 근거: 이미 10절에서 인용한 자동화 워크플로 엔진들은 사람용 칸반보다 훨씬 많은 상태를 쓴다. Airflow^[28]의 TaskInstanceState는 13개(예: UPSTREAM_FAILED를 FAILED와 별도로 구분), BPMN 2.0^[29] 표준의 액티비티 생애주기는 약 10~11개, Temporal^[30]의 WorkflowExecutionStatus는 7~8개다. 전부 사람이 아니라 프로그램(스케줄러)이 다음 행동을 판단해야 해서 "실패"와 "의존성 실패"처럼 사람에게겐 대개 같아 보이는 상태를 구분한다.

이 문서의 결론: 둘은 실제로 충돌하지 않는다 — 대상이 다를 뿐이다. backlog.md의 3상태는 사람이 보는 보드이므로 그대로 유지한다. 06절 오케스트레이터가 필요로 하는 더 세밀한 구분은 새 저장 상태를 만드는 대신, backlog.md가 이미 읽기 시점에 계산해 내려주는 readiness(isReady, isBlocked, blockingDependencies, missingDependencies)를 그대로 쓴다 — 이미 "의존 태스크가 안 끝나서 막힘"과 "의존 태스크 자체가 안 보여서 막힘"을 구분해서 준다. 필요하다면 그 위에 오케스트레이터 자신의 계산된(비저장) 레이어를 엮는다 — Airflow가 TaskInstanceState(태스크 단위, 세밀함) 위에 더 성긴 DagRunState(실행 단위)를 따로 두는 것과 같은 층위 분리다. backlog.md 자체의 저장 상태 어휘를 늘리지 않는다.

09 이해결 설계 질문

실행 기반은 결정됐고, 판단 기준만 남았다

1. Reviewer의 pass/fail 기준, Verifier·AC 판정 기준 — 본인 PKM 완성 후 명세. 출력 스키마와 Verifier·DoD는 이미 07절에서 결정됨.

2. 세션-역할 식별의 세부 방식 — 독립 프로세스라는 큰 방향은 확정(06절)이나, worktree 분리 / read-only 파일권한 등 구체적 구현 방식은 아직 없다.
3. Logger의 정성적 기록 구현 위치 — 토큰 계량은 확정(06절)이나, recap 같은 정성적 기록을 어디에 어떻게 쓸지는 없다.
4. 마일스톤(이상) 단위 에이전트가 정확히 무엇을 집계·검증하는가 — 매니저가 아니라 집계자라는 역할은 확정(05절)이나, 구체적 검증 항목은 예시(fig.6)뿐이다.
5. 다중 LLM 프로바이더 통합에 별도 라이브러리가 필요한가 — Agent Spawner가 프로세스만 스폰한다면 JVM이 LLM API를 직접 부를 이유가 없고, 각 프로바이더의 자체 CLI를 그대로 띄우면 된다. 맞다면 이 항목 자체가 사라지지만, 확인되지 않았다.
6. 헤드리스 세션 간 메시징의 실환경 확인 — Claude Code의 세션 간 메시징 자체는 기본 기능으로 확인됐으나, `claude -p` 헤드리스 프로세스도 inbox 소켓을 바인딩한다고 공식 문서가 명시한다(bare 모드는 예외)^[25]. 남은 것은 실환경 확인뿐이다. 터미널 두 곳에서 헤드리스 프로세스를 띄우고 세 번째 세션에서 `/list-agents`로 잡히는지 보면 30분 안에 확인된다(11절).
7. 보드 상태 감지 방식 — backlog CLI 폴링 vs 파일시스템 WatchService.

10 관련 연구 · 실사례

소프트웨어·에이전트 엔지니어링 문헌과 실제 프로덕션 사례를 조사한 결과, 실사용 증거가 있는 것 위주로, 위 설계 결정과 직접 연결되는 순서로 정리했다.

이 RFC가 속한 분야

Field: Agentic AI(Andrew Ng, 2024~2026 — Gartner 2026 Agentic AI Hype Cycle과도 수렴)

· **Practice: Multi-Agent Collaboration** — Ng이 정의한 4대 agentic 패턴(Reflection/Tool Use/Planning/Multi-Agent Collaboration) 중 하나. Ng의 정의 원문 "서로 다른 역할을 맡은 여러 LLM 인스턴스를 만들어 문제를 푸는 것"은 Implementer/Reviewer/Verifier/Logger 구조를 거의 그대로 서술한다. Ng이 이 패턴의 레퍼런스로 직접 언급하는 구현체는 ChatDev·AutoGen·MetaGPT(CrewAI·LangGraph도 같이 언급하지만, CrewAI는 아래에서 반대 근거로도 쓰인다). Ng 본인이 같은 글에서 이 패턴의 결과 품질이 예측하기 어렵다고 직접 경고하며, 특히 에이전트를 자유롭게 상호작용하게 둘 때를 지목한다 — 이 설계가 자유 대화형이 아니라 고정 순서·고정 트리거 파이프라인(fig.5)인 이유가 여기 있다(00절).

Ng 본인이 만든 OpenWorker^[4]는 정확히는 "specialist variant를 갖는 단일 주 에이전트"이지 역할 분리형은 아니다. 그 감사 추적(audit trail: 누가 왜 그 행동을 했는지 기록) 계층이 Logger의 선례가 된다. 재사용 가치는 GUI 껍데기가 아니라 그 아래 멀티 프로바이더 추상화(aisuite)에 있는데, 그마저도 06절처럼 Spawner가 프로세스만 스폰한다면 필요 없을 수 있다.

이 실행모델의 학술적 계보는 Ng의 2024년 명명보다 훨씬 오래됐다. Wooldridge와 Jennings는 1995년 논문에서 자율성·사회성·반응성·능동성을 에이전트의 필요조건으로 제시했다^[26] — 이 기준의 "사회성"(다른 에이전트와 협상·조율하는 능력)을 엄격하게 적용하면, 지금의 LLM 기반 파이프라인이 실제로 고전적 Multi-Agent Systems(MAS)의 조건을 만족하는지는 여전히 학계에서 논쟁 중이다. 그래서 이 RFC의 4-role 구조는 고전 MAS 이론의 직계 후손이라기보다, Ng이 명명한 현대적 "Agentic AI" 실무 계보에 더 가깝다고 보는 게 정확하다.

보조: Chip Huyen^[5]는 더 넓은 상위 용어로 "AI Engineering"을 쓰며, 본인 책에서 "에이전트는 아직 확립된 이론 틀이 없는 신생 분야"라고 인정한다. 학술 쪽에서 가장 인용 가능한 이름은 "Agentic Software Engineering" — arXiv:2509.06216 로드맵 논문^[6]과 IEEE Xplore 체계적 문헌리뷰^[7]가 있다.

사례	실사용 증거	이 설계와의 관계
Anthropic 오케스트레이터-워커 / 평가자-최적화자 ^[8]	Anthropic 자체 프로덕션 패턴	evaluator-optimizer가 Implementer↔Reviewer와 거의 동일 구조 — 채점기준(rubric)을 명시해야 한다는 전제 조건이 09절 1번의 근거다. 같은 글이 "컨텍스트 공유·의존성 많은 도메인엔 멀티에이전트가 안 맞는다"고도 명시하는데, 이 설계가 그 조건에 해당한다는 점은 00절에서 정면으로 다룬다.
Claude Research 멀티에이전트 ^[1]	실제 출시 가능. 채팅 대비 토큰비용 ~15배, 단일 에이전트 대비는 별도 자료 기준 3~10배. 90.2%는 리서치 평가에서 Opus 리드+Sonnet 서브에이전트가 단일 Opus를 상회한 상대 점수 차다.	Anthropic 권장치(단순조사 1개·비교 2~4개·복잡작업 10개+)는 태스크 복잡도에 따라 에이전트 수를 정하는 06절 Allocation Policy의 실제 선례다. 비용 모델에는 3~10배를 기준으로 쓴다. 본인 태스크 1개를 단일세션 vs 4-프로세스로 git-format 방식으로 직접 비교 측정하면 더 정확하다.
CrewAI hierarchical process	공개된 벤치마크 수치 없음	계층 위임은 위임 1건마다 LLM 왕복이 하나 추가되는 구조적 특성만 근거로 쓴다 — "분당 티켓 처리량"과 사람 승인 게이트가 낀 "일당 태스크" 파이프라인은 단위 자체가 달라 전이되지 않는다. 처리량은 이 설계의 목적 함수가 아니다.
MetaGPT prerequisite gating ^[9]	ICLR 2024 게재. 실증 범위는 HumanEval/MBPP와 소규모 그린필드 앱(SoftwareDev)뿐 — 기존 저장소 수정·회귀 리스크 환경 검증은 없다.	앞 역할의 산출물이 존재해야 다음 역할이 시작하는 구조는 fig.5와 같은 패턴이다("그린필드 소규모 과제에서 검증된 패턴"으로 한정). 더 중요한 기여는 역할 간 산출물이 자유 텍스트가 아니라 구조화된 문서라는 점 — 07절 Verifier 출력 스키마의 근거다.
Cohesion-Aware Task Partitioning	arXiv:2606.00953. 정적분석 기반 파티셔닝으로 2.10배 속도향상, 결합	04-0절의 "파일/모듈 결합도까지 반영한 그래프"가 바로 이 방식이다 — 선언된 의존성 엣지만으로는 부족하다

사례	실사용 증거	이 설계와의 관계
for Multi-Agent Coding ^[10]	이 심하면 자동으로 순차실행에 수렴.	는 가장 직접적인 근거.

JVM 오케스트레이터 참고 사례	실사용 증거	시사점
Akka / Apache Pekko ^[11]	슈퍼비전 트리 기반 액터 프레임워크	프레임워크 채택 여부와 무관하게 슈퍼비전 개념(자식 스폰 실패 시 정책)만 아이디어로 차용했다 — 07절의 비정상종료 시 재스폰 정책이 그 결과다.
Temporal (Java SDK) ^[12]	산업 표준 durable-execution 엔진	워커가 태스크큐를 폴링하는 durable 구조 — 오케스트레이터 재시작 시 인메모리 큐가 유실될 위험에 대한 실제 해법(내구성이 필요해질 때).
Netflix Conductor ^[13]	Netflix 내부 월 ~4.2억 workflow 실행, 150개 앱·20만 workflow 정의 (InfoQ 2026-09)	JVM 네이티브 + 큐 기반. conductor-oss는 LangGraph/CrewAI/OpenAI SDK 에이전트를 durable 엔진 위에서 돌리는 것으로 스스로를 규정한다 — 내구성이 필요해질 때의 참고 후보.
Java virtual thread + Semaphore (JVM 25)	이 설계의 동시성 규모(한 자릿수~십수 개의 OS 서버 프로세스)에 맞는 공개 벤치마크는 없음	06절에서 밝혔듯 근거는 성능이 아니라 코드 단순성이다. Semaphore 상한은 c <code>laude</code> 프로세스의 peak RSS와 API 플랜의 분당 요청 한도로 정한다.
GitHub Actions / GitLab Runner	실제 대규모 프로덕션(수백만 job/일)	bounded pool+큐 패턴이 대규모로 검증됐다. GitLab의 "IdleCount(미리 데워둔 유휴 워커)"는 콜드스타트 지연을 줄이는 완화책으로, 지연 민감 경로를 분리하는 06절 방향에 참고할 만하다.

스케줄링·프로세스 문헌

Evaluating Classical Software Process Models as Coordination Mechanisms for LLM-Based Software Generation^[14](arXiv:2509.13942, 후속 arXiv:2511.09794^[15])는 Waterfall/Scrum 같은 고전 SE 프로세스 모델 자체를 LLM 에이전트 코드생성의 조율 구조로 평가한다 — 일반 DAG 스케줄링이 아니라 SE 프로세스 관점에서 이 RFC와 가장 가까운 동시대 연구다. Coordination as an Architectural Layer for LLM-Based Multi-Agent Systems^[16](arXiv:2605.03310)는 조율 구조를 하드코딩이 아니라 교체 가능한 선언적 계층으로 다루자고 제안한다 — 06절의 "정책을 외부에서 주입"과 독립적으로 같은 결론에 도달한 사례다.

PMBOK 8판(2025-11-13 출간, 6원칙+7성과영역+40개 비규범적 프로세스)은 순차/의존성 관리를 Schedule 성과영역에 두지만 CPM/HEFT 계열을 넘어서는 새 시퀀싱 기법은 없다. AI는 원칙이 아니라 부록(Appendix X3)과 도구 수준으로만 등장하고, "AI가 사람 PM의 추정·일정수립을 보조"하는 관점이 아니라 "AI 에이전트가 실제 작업 인력"인 이 RFC의 시나리오와는 맞지 않는다 — "Is PMBOK Guide the Right Fit for AI?"^[17](arXiv:2506.02214)가 이 간극을 직접 다루며 부정적으로 결론짓는다. PMBOK 8판은 이 설계를 정당화하는 근거로 쓰지 않는다.

Critical Chain Project Management(Eliyahu Goldratt, 1997)와 Kanban의 WIP 제한은 각각 자원 제약 이론과 John Little의 Little's Law라는 서로 다른 전통에서 06절의 "공유 버퍼 하나·동시 작업량 제한"과 같은 결론에 도달한다.

인용 신뢰도에 대한 실용적 교훈

이 문서의 학술 인용(arXiv 2509.06216, IEEE 11343819, MetaGPT ICLR 2024 등 DOI/arXiv ID가 붙은 것)은 전부 원문 대조로 확인됐다. 반면 블로그·마케팅 페이지에서만 확인되던 수치(예: 특정 프레임워크의 처리량 벤치마크, 특정 기업의 도입 사례명)는 검증 과정에서 근거가 없는 것으로 드러나 이 문서에서 전부 뺐다. **DOI나 arXiv ID가 붙은 인용은 신뢰하고, 블로그발 숫자는 항상 원문을 직접 열어본다는 원칙을 이 문서 작성 내내 적용했다.**

여기에 하나를 덧붙인다 — 원문이 맞아도 재는 것이 다르면 인용하지 않는다. 14절에는 멀티에이전트 조율의 손익 임계값(arXiv:2512.08296^[22])과 그 반대쪽 사례(arXiv:2505.16979^[23])를 정량 근거로 싣고 있었다. 원문 대조에서 판본마다 계수가 바뀌는 것을 확인했고(같은 논문의 임계 효과가 벤치마크를 넓히자 절반 가까이 약해졌다), 더 결정적으로 **두 연구가 재는 것은 과제 정확도인데 이 설계가 내세우는 것은 판단의 독립성**이라 종속변수가 달랐다. 그래서 수치를 고치는 대신 인용 자체를 14절에서 뺐다. 두 항목은 기록을 위해 각주에만 남긴다.

11 설계 트레이드오프

다른 멀티에이전트 프레임워크 대비 이 설계가 약한 지점 세 가지와, 각각에 대해 실제로 할 수 있는 것.

① 에이전트 간 직접 메시징

AutoGen류의 실시간 conversable-agent 대화가 이 설계엔 없다. 대신 Claude Code가 세션 간 메시징을 정식 기능으로 제공한다(ListAgents/SendMessage, 머신 로컬 per-session 소켓, Anthropic 서버 비경유)^[25] — 실험적 Agent Teams와 달리 기본 기능이다. 이 도구 설명은 메시지가 "한 Claude가 다른 Claude에게 쓰는 순수 텍스트이며, 보내는 쪽의 대화 이력이나 파일은 절대 포함하지 않는다"고 명시한다.

그래서 프로토콜을 이렇게 제약한다: **메시지에는 포인터만 심는다**(태스크 ID·AC 번호·파일·라인) — 본문은 수신자가 파일에서 직접 읽는다. 4개 에이전트가 할당되는 시점에 이 메시징도 함께 배선하되, 파일 기반

영구 기록(WAL)이 여전히 1차 기록이고 메시징은 실시간 포인터 전달용 보조 채널이다.

06절 Agent Spawner가 스폰하는 `claude -p` 헤드리스 프로세스가 이 메시징의 inbox를 여는지는 공식 문서가 직접 답한다 — 헤드리스 세션도 대화형과 똑같이 inbox 소켓을 바인딩하며 목록에 나타나고, 다만 bare 모드에서는 소켓을 바인딩하지 않아 메시지를 받지 못한다^[25]. 따라서 Spawner는 bare 모드를 쓰지 않는다. 문서상 근거는 확보됐고 실환경 확인만 남았다(09절). 그래도 안 되면 폴백으로 Logger가 이미 보고 있는 WAL 디렉토리를 파일 메일박스로 재사용한다(폴링).

② 비용과 콜드스타트

독립 프로세스 구조상 매번 조상 문서를 루트까지 재로딩하는 비용을 치른다. 핵심 완화책은 **프롬프트/프리픽스 캐싱**을 이 조상 문서 경계에 적용하는 것이다 — `task`→`milestone`→`root` 체인을 한 번 캐시에 쓰면 그 프리픽스를 공유하는 모든 스폰 에이전트가 정상 비용의 ~2.5~10%로 읽는다. 이는 Anthropic이 자체 출시한 기능(Prompt Caching)이며, 별도 평가 연구(arXiv:2601.06007^[18])가 41~80% 비용 절감과 13~31% 지연 감소를 정량 보고한다. 여러 에이전트가 같은 프리픽스를 공유하는 상황 자체를 인프라 레벨에서 푸는 사례로 EACC^[19](Springer)와 SGLang RadixAttention^[20]도 있다. 04절에 명시했듯, 이 캐싱이 성립하려면 조상 문서 프리픽스가 정적이어야 한다.

구조 자체의 정당성도 별도로 확인된다: npj Health Systems(2026, Mount Sinai)^[21]의 임상 규모 실험에서 오케스트레이션된 멀티에이전트가 단일 지속 에이전트 대비 최대 65배 적은 토큰을 쓰면서 정확도도 유지했다(단일 에이전트는 정확도가 붕괴했다) — "독립 프로세스 + 역할분리"라는 이 RFC의 근본 구조가 이 증거로 뒷받침된다.

정확히 "백로그 계층을 루트까지 재로딩"하는 이 시나리오를 1:1로 다룬 논문은 없다 — 더 넓은 "장기맥락·멀티에이전트 컨텍스트 재사용" 문제의 특수 사례로 본다. 04절의 "4개 공통으로 루트까지 전부 읽음" 규칙도 재검토 여지가 있다 — Reviewer/Logger가 Implementer의 요약+diff만 읽는 비대칭 로딩이 가능하다면 고정비를 더 낮출 수 있다(확정 아님, 열린 아이디어).

③ Reviewer / Verifier 채점 기준

MetaGPT의 QA 역할, ChatDev의 Tester 역할은 이미 나름의 평가 기준을 갖고 있다. 이 RFC는 "무엇이 pass/fail인가"가 아직 없다 — 이걸 본인 PKM 완성 후에만 채울 수 있고, 추측으로 채우지 않는다.

다만 PKM과 무관하게 지금 결정 가능했던 것들은 이미 07절에 반영했다: AC 루프의 무한반복 방지 (Reviewer 최대 N회 왕복 후 사람 호출), Verifier 출력 스키마, Verifier를 DoD(결정적 검사, 오늘 구현 가능)와 AC(판단, PKM 대기)로 분리하는 것. 이 세 가지는 "PKM 완성 후"라는 전제와 무관한 순수 엔지니어링 공백이었다.

~/githubs에 실제로 있는 코드를 기준으로, 이 설계에서 무엇이 이미 있고 무엇이 완전히 새로 만들어야 하는지 구분했다.

컴포넌트	상태	비고
langtoolchain	완성 · 즉시 사용 가능	JVM(Java) 런타임 설치 자체가 목적 — 오케스트레이터 부트스트랩에 그대로 사용
git-format	완성 · 상당 부분 재사용	토큰 측정 로직(post-commit)이 이미 구현 돼 있어 Logger 설계의 출발점으로 그대로 가 져다 쓴다
claude-rails	완성 · 콘텐츠 레 이어의 70~80% 기 구 현	브리핑→draft→promote, task view 필독, In Progress+브랜치 확인, 테스트 게이트 커 밋, Done+summary 게이트가 이미 흑으로 동작 중이다. 지금은 "역할 하나" 전제로 만들 어져 있어 4-role 분리에 맞게 확장이 필요하 다.
backlog.md	외부 성숙 도구 · 즉시 사용	계층/상태 모델을 그대로 사용, 신규 구현 없음
context-hub (chub)	완성 · 즉시 사용 가능	외부 API 문서 조회 + 세션 간 학습 (annotation)으로 /clear의 빈틈을 보완한 다
openworker (fork)	참고용 · GUI 껍 데기는 재사용 불 가	GUI 데스크톱 앱이라 헤드리스 임베딩이 안 된다. 재사용 가치가 있는 건 껍데기가 아니라 그 아래 aisuite(멀티 프로바이더 추상화) 인데, 그마저도 Spawner가 프로세스만 스폰 한다면 필요 없을 수 있다(09절).
JVM 오케스트레이터 (Watcher/Queue/Policy/Spawner/Registry)	전혀 존재하지 않 음 — 완전 신규	이 프로젝트에서 유일하게 처음부터 만들어야 하는 부분
에이전트 간 메시징	세션 간 메시징 기능 자체는 존 재. 헤드리스도 inbox를 바인딩 한다고 문서에 명 시(bare 모드 제	ListAgents/SendMessage는 기본 기 능이다. 06절 Spawner가 스폰하는 claude -p 헤드리스 프로세스가 inbox를 여는지는 30분 실측이 필요하다(09절).

컴포넌트	상태	비고
	외) — 실환경 확 인만 남음	

난이도는 실행 기반과 판단 기준으로 갈린다

- 확정된 것은 전부 실행 기반(언어·동시성·프로세스 격리·정책 주입 방식)이고, 미결인 것은 전부 이 시스템
 이 실제로 무엇을 검증하는가(Reviewer/Verifier 채점 기준)다. 그래서 난이도 추정도 둘로 나눈다.
- Walking skeleton(지금 바로 가능) — 며칠.** Implementer 1개만 스폰 → 토큰 집계(git-format 방식 재사용)
 → /clear → 회수. Semaphore·Spawner·Registry·토큰 측정을 한 번에 검증하면서, 아직 없는 역할 명세를
 전혀 요구하지 않는 유일한 조각이다 — 다음에 할 일은 이것이다.
 - 4-role 전체 — 추정 불가.** Reviewer/Verifier·AC 채점 기준이 없는 한 "완성"의 정의 자체가 없어 기간을 추정할
 근거가 없다.
 - claude-rails를 역할 인식형으로 확장 — 중간.** 기존 28개 혹은 role별로 분기하는 작업이다. 새로운 개념은 아니고
 기존 코드 이해가 관건이다.
 - 세션 간 메시징 통합 — 낮음, 단 헤드리스 실측이 전제.** 실측에서 되는 걸로 확인되면 이미 있는 기능을 쓰기만 하면
 된다. 안 되면 폴백 구현(파일 메일박스)이 필요해 난이도가 올라간다.

결론: 제일 어려운 건 코드가 아니라 아직 정해지지 않은 판단 기준(Reviewer/Verifier 채점기준)이다. 실행
 기반 결정은 이미 끝났고, 이미 코드로 존재하는 부분(claude-rails/git-format/
 backlog.md/langtoolchain/chub)의 구현 리스크는 낮다.

13 이 문서에서 가장 견고한 근거

독립적으로 검증했을 때도 흔들리지 않은 부분들 — 나머지 절의 불확실성과 대비된다.

- 06절 토큰 측정** — 원본 코드(git-format)를 직접 읽고, 델타 커서·실패·투명성 패턴을 구체적으로 가져왔으며, 원
 저자의 자기 한계 고백까지 반영해 확정하지 않고 미정으로 남겼다.
- 04절 AC vs DoD 구분** — 개념적으로 정확하고 backlog.md 기능으로 뒷받침되며, 07절 Verifier 분리(DoD/AC)
 의 근거가 된다.
- 02절 부트스트랩 순서** — 인용 없이 실제 의존성(CLI 없이는 명령이 존재하지 않음)에서 직접 도출된다.
- 06절 "독립 프로세스가 토큰 귀속을 쉽게 만든다"** — 외부 인용 없이 스스로 선 논증이고, 실제로 맞다.
- 학술 인용 3건**(arXiv 2509.06216 / IEEE 11343819 / MetaGPT ICLR 2024) — 전부 원문 대조로 확인됐다
 (10절의 인용 신뢰도 교훈 참고).

- 03절 fig.3-1 재진입 조건 — 실제 claude-rails 운영 규칙과 일치하고, 조용한 스코프 확장을 구조적으로 막는다.

14 적용 범위 — 판단 단위는 프로젝트가 아니라 태스크

이 절이 확인하려는 것은 "이 프로젝트 종류엔 맞는가"가 아니라 "이 태스크엔 맞는가"다. Next.js 스캐폴드나 Spring+DB 단순 CRUD는 그 질문에 답하기 위한 예시일 뿐, 프로젝트 전체가 영구히 4-role 대상에서 제외된다는 뜻이 아니다 — 같은 프로젝트 안에서도 태스크 난이도가 올라가면 그 태스크부터 4-role을 쓴다. 그 난이도의 경계선을 외부 수치로 정하려던 시도는 아래처럼 철회했고, 지금은 측정 대상으로 남겨둔다.

경계선은 빌려오지 않는다

"얼마나 어려워야 4-role을 쓰는가"에 대한 정량적 임계값을 외부 연구에서 가져오려 했고, 한동안 이 자리에 특정 수치를 적어뒀다. **뻘뻘다**. 멀티에이전트 조율의 손익을 다루는 연구들이 재는 것은 과제 정확도이고, 이 설계가 내세우는 것은 판단의 독립성이라 **종속변수가 다르다**. 정확도 기준으로 "이득 없음"이 나와도 회귀 방지 가치는 별도로 남고, 반대로 정확도가 올라가도 그것이 독립 판정 덕이라는 보장이 없다.

운용상으로도 쓸 수 없다. 배정 시점에 "이 태스크의 단일 에이전트 기준선 성공률"은 알 수 없고, 벤치마크 평균 정확도는 눈앞의 이 태스크의 성공률이 아니다. 게다가 소프트웨어에서 놓친 회귀의 비용은 검증에 드는 토큰·시간과 비대칭인데, 평균 성능을 보는 지표에는 그 비대칭이 들어 있지 않다.

그래서 이 문서는 임계값을 명시하지 않는다. 판단 기준은 00절의 반증 가능한 중단 기준 하나이고, 경계선은 12절 walking skeleton이 쌓는 태스크별 성공/실패·재작업·회귀 기록으로 **이 저장소의 도메인에서 직접 측정해** 정한다. 남의 임계값을 상수로 박아두는 것보다 이쪽이 이 문서의 성격에도 맞는다.

"복잡한가"를 재는 잣대

"On the Importance of Task Complexity..."^[24](arXiv:2510.04311)는 깊이(depth, 추론 길이) × 너비(width, 필요 역량의 다양성) 두 축으로 복잡도를 형식화한다 — "복합 소프트웨어 프로젝트"를 막연히 말하는 대신 쓸 수 있는 실제 잣대다. Next.js/Spring+DB 스캐폴드는 두 축 모두 낮게 나올 것으로 예상된다. 깊이와 너비라는 두 축은 04-0의 스케줄 계측(임계 경로 길이와 그래프 폭)과도 같은 형태다.

정직한 빈틈

- 소프트웨어공학의 "프로세스 오버헤드 대 프로젝트 규모" 문헌(Agile tailoring 등)은 "규모에 맞게 조정하라"는 원칙 자체는 정설이지만, 정량적 임계 모델은 존재하지 않는다 — 정성적 원칙 이상으로 확대 해석하지 않는다.
- ChatDev/MetaGPT/AutoGen류 파이프라인을 말 그대로 trivial한 CRUD 스캐폴드에 직접 돌려 보고한 연구는 찾지 못했다 — 이 대목은 근거 없이 상식에 기대고 있다.

정리하면 이 절에는 빌려온 숫자가 없다. 작고 단순한 프로젝트는 Implementer 단독으로 시작하고, 4-role 전체는 단일 에이전트가 자주 실패하는 더 복잡한 작업에 아껴 쓴다 — 그 "자주"의 경계선은 00절의 반증 기준과 12절 walking-skeleton의 실측이 채운다.

각주 인용 • 출처

- [1] Anthropic, “How we built our multi-agent research system” — <https://www.anthropic.com/engineering/multi-agent-research-system>
- [2] Andrew Ng, “Agentic Design Patterns Part 5: Multi-Agent Collaboration”, *The Batch* (DeepLearning.AI) — 본문의 “결과 품질을 예측하기 어렵다” 인용 출처. 코스 페이지가 아니라 이 글이 원문이다. <https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-5-multi-agent-collaboration>
- [3] Claude Code Docs, “Agent Teams” — <https://code.claude.com/docs/en/agent-teams>
- [4] Andrew Ng, OpenWorker (GitHub) — <https://github.com/andrewyng/openworker>
- [5] Chip Huyen, “Agents” — <https://huyenchip.com/2025/01/07/agents.html>
- [6] “Agentic Software Engineering: Foundational Pillars and a Research Roadmap”, arXiv:2509.06216 — <https://arxiv.org/abs/2509.06216>
- [7] “Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review”, IEEE Xplore — <https://ieeexplore.ieee.org/document/11343819/>
- [8] Anthropic, “Building Effective Agents” — <https://www.anthropic.com/engineering/building-effective-agents>
- [9] MetaGPT, ICLR 2024 — https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf
- [10] “When Parallelism Pays Off: Cohesion-Aware Task Partitioning for Multi-Agent Coding”, arXiv:2606.00953 — <https://arxiv.org/abs/2606.00953>
- [11] Apache Pekko (프로젝트 사이트) — <https://pekko.apache.org/>
- [12] Temporal Docs, “Task Queues” — <https://docs.temporal.io/task-queue>
- [13] Conductor OSS (Netflix Conductor) — <https://conductor-oss.org/>
- [14] “Evaluating Classical Software Process Models as Coordination Mechanisms for LLM-Based Software Generation”, arXiv:2509.13942 — <https://arxiv.org/abs/2509.13942>
- [15] “Evaluating Software Process Models for Multi-Agent Class-Level Code Generation”, arXiv:2511.09794 — <https://arxiv.org/abs/2511.09794>
- [16] “Coordination as an Architectural Layer for LLM-Based Multi-Agent Systems”, arXiv:2605.03310 — <https://arxiv.org/abs/2605.03310>
- [17] “Is PMBOK Guide the Right Fit for AI?”, arXiv:2506.02214 — <https://arxiv.org/abs/2506.02214>

- [18] “Don’t Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks”, arXiv:2601.06007 — <https://arxiv.org/abs/2601.06007>
- [19] “EACC: Efficient Agent Context Cache Sharing for Multi-Agent Systems”, Springer — https://link.springer.com/chapter/10.1007/978-981-95-8405-5_9
- [20] SGLang / RadixAttention, arXiv:2312.07104 — <https://arxiv.org/abs/2312.07104>
- [21] npj Health Systems (Nature Portfolio), Mount Sinai 임상 규모 실험 — <https://www.nature.com/articles/s44401-026-00077-0>
- [22] “Towards a Science of Scaling Agent Systems”, arXiv:2512.08296 — <https://arxiv.org/abs/2512.08296>
본문 인용 없음 — 종속변수 불일치로 14절에서 제외(10절 기록).
- [23] “Know the Ropes: A Heuristic Strategy for LLM-based Multi-Agent System Design”, arXiv:2505.16979 — <https://arxiv.org/abs/2505.16979>
본문 인용 없음 — 종속변수 불일치로 14절에서 제외(10절 기록).
- [24] “On the Importance of Task Complexity in Evaluating LLM-Based Multi-Agent Systems”, arXiv:2510.04311 — <https://arxiv.org/abs/2510.04311>
- [25] Claude Code Docs, “Cross-session messaging” 및 “Agent teams” — 본문의 세 주장(머신 로컬 per-session 소켓 · Anthropic 서버 비경유, 메시지는 순수 텍스트로 보내는 쪽 대화 이력·파일 미포함, Agent Teams는 실험적이나 세션 간 메시징은 기본 기능)을 공식 문서에서 확인했다. <https://code.claude.com/docs/en/cross-session-messaging>
- [26] M. Wooldridge & N. R. Jennings, “Intelligent Agents: Theory and Practice”, *The Knowledge Engineering Review*, 1995 — 안정적인 공개 URL은 확인되지 않았다. (링크 없음)
- [27] MrLesk, Backlog.md — README 및 CLI-INSTRUCTIONS.md — <https://github.com/MrLesk/Backlog.md>
- [28] Apache Airflow Docs, “Tasks” (TaskInstanceState) — <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/tasks.html>
- [29] OMG, Business Process Model and Notation (BPMN) 2.0 명세 — <https://www.omg.org/spec/BPMN/2.0/>
- [30] Temporal Docs, “Workflow Execution Status” — <https://docs.temporal.io/workflows#status>
- [31] David J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*, Blue Hole Press, 2010, ISBN 978-0984521401 — 안정적인 공개 URL은 확인되지 않았다. (링크 없음)